OFFSIDE LABS

Dynamic Bonding Curve

0.1.8

Smart Contract Security Assessment

January 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **6**

4 Key Findings and Recommendations **7**

4.1 Missing Validation on Config Parameters May Lead to Migration Failure 7

4.2 Possible Division-By-Zero May Lead to Migration Failure 8

4.3 Informational and Undetermined Issues 9

5 Disclaimer **11**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Dynamic Bonding Curve* smart contracts, starting on December 21th, 2025, and concluding on December 31th, 2025.

Project Overview

The Dynamic Bonding Curve (DBC) is a permissionless launch pool protocol that enables users to create and launch tokens directly with customizable bonding curves. The program also allows partners to configure key parameters such as quote tokens, token graduation curves, and fees.

Release 0.1.8 introduces a configurable pool creation fee mechanism and adds dedicated endpoints for partners and the protocol to claim their respective shares of the creation fee. It also adds LP vesting support for DAMM v2 migration, allowing configurations to specify post-migration liquidity lock/vesting parameters that are used to execute the locking process during migration.

Audit Scope

The assessment scope contains mainly the smart contracts of the dynamic-bonding-curve program for the *Dynamic Bonding Curve* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Dynamic Bonding Curve
  - Codebase: `https://github.com/MeteoraAg/dynamic-bonding-curve`
  - Release 0.1.8: `PR-151`
  - Commit Hash: 5b6238e93c2d466a8b955ad781386d6fd7d6c528

The issues described in this report have been resolved by the following commit:

- Dynamic Bonding Curve
  - Codebase: `https://github.com/MeteoraAg/dynamic-bonding-curve`
  - Commit Hash: e62727a9284acf800fe56c5e994912f960641997

We listed the files we have audited below:

- Dynamic Bonding Curve Release 0.1.8
  - programs/dynamic-bonding-curve/src/access_control.rs
  - programs/dynamic-bonding-curve/src/constants.rs
  - programs/dynamic-bonding-curve/src/error.rs
  - programs/dynamic-bonding-curve/src/event.rs
  - programs/dynamic-bonding-curve/src/instructions/admin/ix_close_claim_protocol_fee_operator.rs

- programs/dynamic-bonding-curve/src/instructions/admin/ix_create_claim_protocol_fee_operator.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_claim_creator_trading_fee.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_create_virtual_pool_metadata.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_transfer_pool_creator.rs
- programs/dynamic-bonding-curve/src/instructions/creator/ix_withdraw_creator_surplus.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_spl_token.rs
- programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/damm_v2_utils.rs
- programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/migrate_damm_v2_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/ix_withdraw_migration_fee.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/meteora_damm_claim_lp_token.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/meteora_damm_lock_lp_token.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/meteora_damm_metadata_state.rs
- programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/migrate_meteora_damm_initialize_pool.rs
- programs/dynamic-bonding-curve/src/instructions/migration/withdraw_leftover.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_legacy_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_protocol_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/operator/ix_withdraw_protocol_surplus.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_pool_creation_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_claim_partner_trading_fee.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs
- programs/dynamic-bonding-curve/src/instructions/partner/ix_withdraw_partner_surplus.rs

- programs/dynamic-bonding-curve/src/instructions/swap/ix_swap.rs
- programs/dynamic-bonding-curve/src/lib.rs
- programs/dynamic-bonding-curve/src/math/safe_math.rs
- programs/dynamic-bonding-curve/src/params/fee_parameters.rs
- programs/dynamic-bonding-curve/src/state/config.rs
- programs/dynamic-bonding-curve/src/state/virtual_pool.rs
- programs/dynamic-bonding-curve/src/utils/token.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 2 medium issues
- 0 low issue
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Missing Validation on Config Parameters May Lead to Migration Failure	Medium	Fixed
02	Possible Division-By-Zero May Lead to Migration Failure	Medium	Fixed
03	Incorrect Error Message and Error Type	Informational	Fixed
04	Duplicate LiquidityVestingInfoParams Validation in create_config IX	Informational	Fixed

4 Key Findings and Recommendations

4.1 Missing Validation on Config Parameters May Lead to Migration Failure

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

The validation of `LiquidityVestingInfoParams` in the `create_config` instruction is not fully consistent with `VestingParameters::validate()` in DAMM v2 `lock_position` . These inconsistencies include:

Scenario 1:

When creating a config in DBC, the following parameter combination is allowed:

- `cliff_duration_from_migration_time == 0 && number_of_periods == 0`

However, in DAMM v2 `lock_position` , the checks shown in the code snippet below will return an error when encountering the parameter combination above.

```
39          require!(cliff_point >= current_point,
                ↪ PoolError::InvalidVestingInfo);
40
41          if cliff_point == current_point {
42              require!(self.number_of_period > 0,
                    ↪ PoolError::InvalidVestingInfo);
43          }
```

programs/cp-amm/src/instructions/ix_lock_position.rs#L39-L43

Scenario 2:

When computing DAMM v2 `VestingParameters` , if `LiquidityVestingInfo` parameter values or `total_vested_liquidity` are too small, and combining with integer division rounds down, the calculated `liquidity_per_period` may be 0, while `number_of_period > 0 && frequency > 0` .

Such parameters will also cause an error in DAMM v2 `lock_position` .

```
45          if self.number_of_period > 0 {
46              require!(
47                  self.period_frequency > 0 && self.liquidity_per_period >
                        ↪ 0,
48                  PoolError::InvalidVestingInfo
49              );
50          }
```

programs/cp-amm/src/instructions/ix_lock_position.rs#L45-L50

Impact

The migration may fail to complete due to an error from `lock_position` in DAMM v2.

Recommendation

It is recommended to add more checks for these parameters, ensuring that when creating the config, the constraints on these parameters are fully consistent with the constraints in the related DAMM v2 instruction.

For cases where `liquidity_per_period` may be 0, it may be handled as follows:

- Downgrade this lock to a cliff-only lock.
- Adjust other parameter values (for example, decreasing `number_of_period`) to ensure `liquidity_per_period ≥ 1` .

Mitigation Review Log

Fixed in the commit 761c37907975a80134b76d05c04f253fa2571c73 and 73bc283877619cce4d483025fa77674cf354cc93.

4.2 Possible Division-By-Zero May Lead to Migration Failure

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

During migration to DAMM v2, the parameters of `second_position` are adjusted via `LiquidityDistributionItem::adjust_liquidity` .

```
1057         let numerator = self.liquidity_vesting_info.vesting_percentage;
1058
1059         let denominator =
                 ↪ numerator.safe_add(self.permanent_locked_liquidity_percentage)?;
1060
1061         let vested_liquidity = safe_mul_div_cast_u128(
1062             liquidity_subject_to_lock,
1063             numerator.into(),
1064             denominator.into(),
1065             Rounding::Down,
1066         )?;
```

programs/dynamic-bonding-curve/src/state/config.rs#L1057-L1066

When computing `vested_liquidity`, if both `vesting_percentage` and `permanent_locked_liquidity_percentage` are 0, a division-by-zero error will occur.

Impact

This division-by-zero error will directly prevent this pool from being migrated, and in this case it also cannot be fixed by modifying the parameter values in `Config`.

Recommendation

It is recommended to handle the case where `vesting_percentage + permanent_locked_percentage == 0` separately.

Mitigation Review Log

Fixed in the commit 6999f2cc078adaf2cbea725cafbf3a976286018e.

4.3 Informational and Undetermined Issues

Incorrect Error Message and Error Type

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the `Safecast` trait, the error message is "Math error". It is recommended to update it to casting-related wording:

```
130                     Err(_) => {
131                         let caller = Location::caller();
132                         msg!("Math error thrown at {}:{}", caller.file(),
                                ↪ caller.line());
133                         Err(PoolError::TypeCastFailed)
134                     }
```

programs/dynamic-bonding-curve/src/math/safe_math.rs#L130-L134

In the `claim_legacy_pool_creation_fee` IX, the error type used in this requirement may also be improper.

```
34      require!(
35          pool.has_legacy_creation_fee() &&
        ↪ pool.eligible_to_claim_legacy_creation_fee(),
36          PoolError::Unauthorized
37      );
```

programs/dynamic-bonding-curve/src/instructions/operator/ix_claim_legacy_pool_creation_fee.rs#L34-L37

Fixed in the commit 0903987ef92396803c159a8827b3c3511876af14.

Duplicate LiquidityVestingInfoParams Validation in create_config IX

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

In the `create_config` instruction, the same validation is performed twice for `ConfigParameters.partner_liquidity_vesting_info` and `ConfigParameters.creator_liquidity_vesting_info`.

First validation:

```
320                 // validate vesting
321                 require!(
322                     self.partner_liquidity_vesting_info.is_zero(),
323                     PoolError::InvalidVestingParameters
324                 );
325                 require!(
326                     self.creator_liquidity_vesting_info.is_zero(),
327                     PoolError::InvalidVestingParameters
328                 );
```

programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs#L320-L328

```
339                 // validate vesting
340                 self.partner_liquidity_vesting_info.validate()?;
341                 self.creator_liquidity_vesting_info.validate()?;
```

programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs#L339-L341

Second validation:

```
386         self.partner_liquidity_vesting_info.validate()?;
387         self.creator_liquidity_vesting_info.validate()?;
```

programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs#L386-L387

It is recommended to remove the second validation.

Fixed in the commit 0903987ef92396803c159a8827b3c3511876af14.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

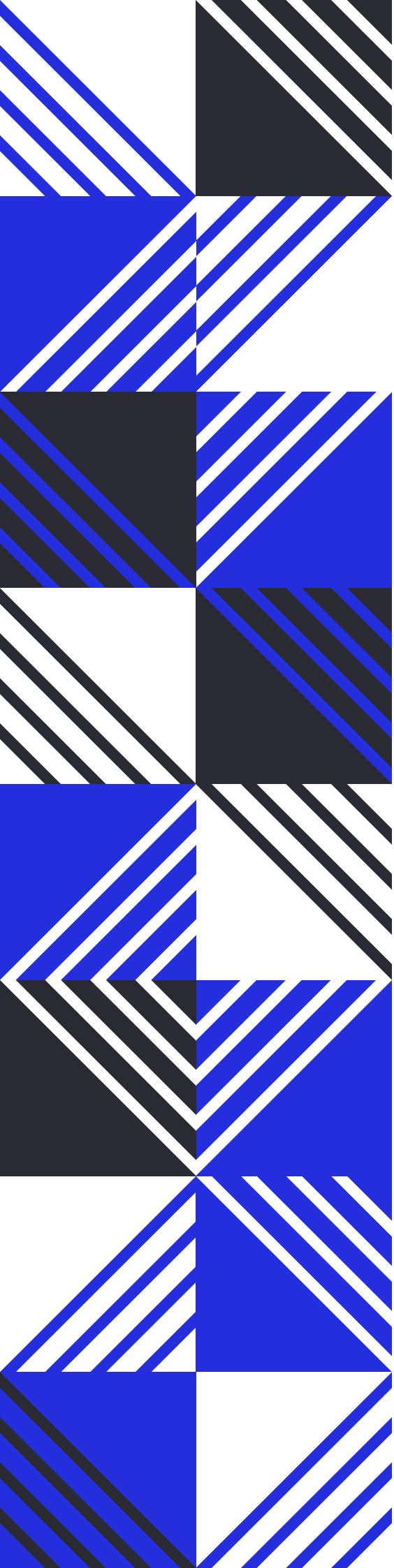
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs